UNIVERSITÀ DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in
Informatica

Elaborato finale

Move Fuzzer

Supervisore
Patrignani Marco

Laureando
Podavini Luca

Anno accademico 2023/2024

Ringraziamenti

Ringrazio il mio relatore Prof. Patrignani per la sua pazienza, la sua grande disponibilità e per il supporto, fondamentale per la stesura dell'elaborato. Una ringraziamento speciale va anche ai ragazzi dell'azienda Mysten Labs, con la quale ho collaborato durante questo progetto. Vi ringrazio per tutte le informazioni e tutti i consigli che mi avete dato, senza i quali non sarei riuscito a completare questo progetto.

Ringrazio la mia famiglia, in particolare i miei genitori e mia sorella Elena, per il grande supporto ricevuto durante questi anni. Mi avete sempre supportato sia nei bei momenti che in quelli di difficoltà, qualsiasi parola non sarebbe sufficiente per ringraziarvi abbastanza.

Ringrazio tutti i miei amici per questi anni passati insieme, per le intense sessioni di studio e soprattutto grazie per tutte le volte che, con il vostro ottimismo, avete detto:"Dai che questo esame è facile!", anche se non sempre è andata come previsto. Un ringraziamento speciale va al gruppo Orazio Gin, grazie per aver dato fiducia ad uno sconosciuto che faceva gare di flessioni in Scaletta; sono stati tre anni indimenticabili. Un ringraziamento speciale va anche ai miei amici di sempre, anche se non avete contribuito direttamente alla stesura di questa tesi, la vostra presenza e il vostro supporto è stato fondamentale.

Ringrazio infine i ragazzi del FabLab, persone stimolanti sempre disponibili per insegnarmi cose nuove e supportarmi nei miei progetti. Grazie a voi ho trovato un ambiente accogliente dove condividere le mia passioni al di fuori dei corsi universitari facendo esperienze fondamentali per il futuro.

Indice

1 Sommario **2**

2 Struttura del progetto **4**
2.1 Interfaccia verso l'utente . . . 4
2.2 Package CLI . . . 4
2.3 Package move-libfuzzer . . . 5
2.4 Move Sui . . . 5
2.5 Configurazione iniziale . . . 6

3 Package move-libfuzzer **8**
3.1 Interazione con libfuzzer tramite FFI . . . 8
3.1.1 Compilazione e linking . . . 9
3.1.2 Inizializzazione . . . 10
3.1.3 Esecuzione . . . 11
3.2 Modulo move-runner . . . 12
3.2.1 Caricamento dei moduli - il modulo move_loader . . . 13
3.2.2 Gestione dei tipi - il modulo types . . . 13
3.2.3 Analisi della funzione target . . . 16
3.2.4 Creazione della MoveVM e di MoveRunner . . . 17
3.2.5 Gestione dei moduli durante il runtime . . . 18
3.2.6 Creazione della sessione . . . 18
3.2.7 Generazione degli input - il modulo arbitrary_inputs . . . 18
3.2.8 Esecuzione dei target . . . 21

4 Package CLI **23**
4.1 Inizializzazione della CLI . . . 23
4.2 Il comando INIT . . . 24
4.3 Il comando ADD . . . 24
4.4 Il comando BUILD . . . 24
4.4.1 Selezione del target . . . 24
4.4.2 Configurazioni del compilatore . . . 25
4.5 Il comando RUN . . . 25
4.6 Il comando FMT . . . 26
4.7 I comandi COVERAGE, CMIN e TMIN . . . 26

5 Valutazione e sviluppi futuri **27**
5.1 Sviluppi futuri . . . 28

6 Conclusioni **30**

Bibliografia **30**

1 Sommario

Il *fuzzing* (o *fuzz-testing*) è una tecnica utilizzata dai programmatori per eseguire dei test sui propri software. Quest'ultima è finalizzata a trovare vulnerabilità e falle di sicurezza utilizzando appositi strumenti, detti fuzzer, i quali eseguono i programmi con input non validi, mal formati o casuali. Monitorando il comportamento durante l'esecuzione è possibile rivelare malfunzionamenti come arresti improvvisi, asserzioni errate o *memory-leaks*.

L'obiettivo di questa tesi è la progettazione e lo sviluppo di un fuzzer per il linguaggio Move, un linguaggio di programmazione per la creazione di *smart-contracts* nato nel 2018 come una parte del progetto Facebook Libra. La sicurezza e l'affidabilità sono fattori cruciali durante lo sviluppo di software basato su *blockchain* e *smart-contracts*, l'impiego dunque di un fuzzer durante la fase di test permette di ridurre notevolmente falle di sicurezza e malfunzionamenti risultando in un prodotto di qualità superiore.

La struttura di questo progetto, nominato *move-fuzz*, è ispirata ad un fuzzer già esistente per il linguaggio Rust, chiamato *cargo-fuzz*, seppur con molte differenze dovute alla diversa natura dei due linguaggi. Rust è in effetti un linguaggio compilato mentre Move viene eseguito all'interno di una macchina virtuale, questo aspetto è fondamentale per comprendere alcune criticità del progetto esposte nel corso della lettura. La differenza principale tra le due implementazioni riguarda proprio come essi vengono compilati e come essi interagiscono con il codice da testare. In Rust il codice del fuzzer viene compilato assieme al codice da testare per formare un unico file eseguibile, in Move invece, il codice del fuzzer viene compilato una sola volta durante l'installazione mentre il codice da testare viene compilato separatamente prima di ogni esecuzione. Questo comporta una minore interazione tra il fuzzer e il codice da eseguire, risultando in una minore efficienza del fuzzer.

Per l'implementazione di *move-fuzz* è stata mantenuta in gran parte l'infrastruttura esistente per Rust, alla quale è stata aggiunta una parte per gestire l'esecuzione del codice in Move. L'implementazione di *cargo-fuzz* è basata su *libfuzzer*, un motore di fuzzing all'interno del progetto LLVM (Low-Level Virtual Machine): una collezione di strumenti e librerie per la compilazione di linguaggi di programmazione. Questa libreria, scritta in C++, viene compilata e integrata dal linker all'interno del fuzzer, il suo compito è quello di generare dati da fornire alle librerie da testare ed eseguirle utilizzando particolari funzioni (dette *fuzz-targets*) come punto d'ingresso. Le differenze principali tra l'implementazione per Rust e quella per Move, stanno, come accennato in precedenza, nelle modalità di interazione tra *libfuzzer* e le funzioni target. Nella versione per Rust, la libreria *libfuzzer*, il codice sorgente del fuzzer e il codice sorgente di ogni target vengono compilati all'interno dello stesso programma eseguibile. I tre componenti sono dunque a stretto contatto durante l'esecuzione a differenza di quello che accade in Move. Nella versione per Move, invece, *libfuzzer* viene compilato una volta sola in fase di installazione e integrato insieme al codice del fuzzer all'interno di un unico eseguibile detto *move-libfuzzer*. Il codice sorgente dei target viene compilato soltanto in fase di esecuzione, i moduli Move vengono caricati e analizzati in un secondo momento da *move-libfuzzer*, non vi è perciò lo stretto legame presente in *cargo-fuzz*. È stata infine sviluppata una Command Line Interface per gestire le richieste dell'utente, effettuare le opportune configurazioni ed infine eseguire *move-libfuzzer* fornendo i dati per identificare e caricare la funzione da testare.

Il progetto è stato verificato sia utilizzando progetti reali scritti in Move che casi di test più specifici. In generale, il fuzzer funziona bene per rilevare errori aritmetici dovuti principalmente ad overflow, è invece meno efficiente per casi di test specifici su stringhe e vettori. Per ovviare a questo problema, il progetto può essere migliorata l'interazione tra il fuzzer e il codice move, al momento abbastanza

distaccati tra di loro. Uno studio più approfondito della copertura del codice potrebbe migliorare significativamente sia l'efficienza che l'efficacia del fuzzer.

Il documento è suddiviso in 5 capitoli. Il capitolo 2, dal titolo "Struttura del progetto", illustrandone i principali componenti all'interno di *move-fuzz* e il loro ruolo all'interno dell'applicazione. Nei due capitoli successivi verrà approfondita l'implementazione dei due componenti principali, *move-libfuzzer* e la CLI, rispettivamente nei capitoli 3 e 4. Nel capitolo 5 sono presenti delle considerazioni sulla valutazione del progetto, sui test che sono stati effettuati e sui possibili sviluppi futuri. Il capitolo 6 contiene infine delle considerazioni personali sul progetto come conclusione della lettura.

2 Struttura del progetto

Il progetto, scritto in Rust, è strutturato in due package[1] principali: l'interfaccia a riga di comando **(CLI)** e *move-libfuzzer*. La CLI si occupa di gestisce le richieste dell'utente attraverso un'interfaccia a riga di comando, il suo compito è quello di apportare le configurazioni necessarie per eseguire il fuzzer su una determinata funzione. Gestisce la compilazione dei moduli Move, configura l'ambiente di esecuzione ed infine esegue *move-libfuzzer* con gli opportuni parametri. Il package *move-libfuzzer*, ha il compito di interfacciarsi con la libreria *libfuzzer* per generare i dati da fornire alle funzioni *target* scritte in Move. Entrambi i *package* richiedono l'accesso alle funzionalità interne del linguaggio Move, è stato quindi necessario importare la toolchain di Move all'interno del progetto. Ciò avviene attraverso `git-submodule`, uno strumento di Git che permette di importare *repository* di terze parti all'interno dei propri progetti.

All'interno di questo capitolo viene descritta la struttura globale del progetto, vengono illustrati i ruoli di ciascun componente e il funzionamento generale del programma senza approfondire i dettagli di implementazione. Vengono inizialmente descritti i package *move-libfuzzer* e *move-cli*, rispettivamente nelle sezioni 2.2 e 2.3. La sezione 2.4, introduce la toolchain di *move-sui* e le funzionalità utilizzate per programmare il fuzzer mentre la sezione 2.5 parla infine della fase di installazione e delle configurazioni preliminari, necessarie per l'esecuzione del programma.

2.1 Interfaccia verso l'utente

L'interazione tra l'utente e *move-fuzz* avviene principalmente attraverso la Command Line Interface e le funzioni target. Tramite la CLI è possibile inizializzare l'ambiente di esecuzione, compilare i casi di test ed infine eseguirli. Le funzioni target, sono invece particolari funzioni scritte in Move che vengono utilizzate come punto d'ingresso per eseguire i casi di test. È importante notare che esistono due modalità per identificare una funzione target, è infatti possibile eseguire il fuzzer su un target vero (identificato tramite il proprio nome) oppure direttamente su alcune funzioni all'interno del codice (identificate tramite il loro nome e il nome del modulo che le contiene). Per poter eseguire direttamente una funzione, deve accettare parametri dei tipi generabili in maniera automatica dal fuzzer. La generazione automatica non è possibile in due casi: quando la funzione richiede l'inizializzazione di alcuni valori o quando la funzione richiede in ingresso dei parametri per riferimento. In questo ultimo caso è obbligatorio istanziare gli oggetti in una funzione target dedicata e passarli in un secondo momento alla funzione da testare. È tuttavia una buona pratica la creazione di target dedicati, soprattutto per mantenere maggiore ordine all'interno del codice e definire con precisione quali funzioni sono dedicate al *fuzzing*.

La CLI permette di eseguire numerose altre operazioni che verranno descritte in modo più dettagliato nella sezione successiva, dedicata alla CLI.

2.2 Package CLI

La CLI è, come accennato in precedenza, lo strumento principale con il quale l'utente può interagire con *move-fuzz*. Il suo compito è fare da intermediario tra l'utente e *move-libfuzzer*, il componente che si occupa effettivamente di eseguire il fuzzer. Nello specifico, la CLI deve configurare l'ambiente di esecuzione, gestire la compilazione dei moduli Move, configurare i parametri di default ed infine richiamare *move-libfuzzer* con i parametri impostati dall'utente. Le operazioni disponibili sono:

- **init**: Inizializza il progetto per il fuzzing.
- **add**: Aggiunge un nuovo target al progetto.

[1] Un *package* è un insieme di uno o più *crates* che fornisce un insieme di funzionalità. Un *package* contiene un file Cargo.toml che descrive la sua struttura.

- **build**: Compila un determinato target del fuzzer.
- **run**: Esegue il fuzzer su un determinato caso di test
- **fmt**: Esegue un determinato caso di test con degli input specifici scelti dal *corpus*.
- **cmin**: Cerca di trovare il più piccolo sottoinsieme di input (*corpus*) che permette di avere la massima copertura del codice. Questa operazione è utile per ridurre i tempi di esecuzione del fuzzer, viene quindi eseguita prima dell'esecuzione dei test.
- **tmin**: Minimizza un caso di test. Quando viene trovato un input che causa un errore all'interno del caso di test, è utile cercare di ridurre la sua dimensione mantenendo invariato il suo comportamento.
- **coverage**: Genera un report sulla copertura del codice.

2.3 Package move-libfuzzer

Il package *move-libfuzzer* ha, come accennato in precedenza, il compito di eseguire il fuzzer vero e proprio seguendo le indicazioni ricevute dalla CLI. Nello specifico, i dati necessari per eseguire una funzione Move sono:

- **target-function**: il nome della funzione da eseguire;
- **target-module**: il nome del modulo che contiene la funzione da eseguire;
- **module-path**: il percorso del file compilato che contiene il *bytecode* del *target-module*.

Tramite queste informazioni, *move-libfuzzer*, carica in memoria il *target-module* seguito da tutte le sue dipendenze, genera i dati da passare alla funzione ed infine la esegue attraverso la Move Virtual Machine. È importante notare come *move-libfuzzer* non sia a conoscenza della distinzione tra target e normali funzioni move. In questo componente del fuzzer le funzioni da eseguire vengono identificate in qualsiasi caso tramite il loro nome e il nome del proprio modulo. È compito della CLI rendere questa logica invisibile da parte di *move-libfuzzer*.

2.4 Move Sui

Per accedere alle funzionalità interne del linguaggio Move e poter implementare le varie funzionalità del fuzzer è stata importata all'interno del progetto la toolchain di *move-sui*. Quest'ultima è stata importata utilizzando `git-submodule`, una funzionalità di Git che permette di integrare all'interno della propria repository del codice di terze parte mantenendo il riferimento alla repository originale. Le funzionalità maggiormente utilizzate per lo sviluppo di *move-fuzz* sono state:

- `move-vm-runtime` e `move-vm-config`: contengono l'implementazione delle Move Virtual Machine e le sue configurazioni. Sono fondamentali per l'esecuzione dei programmi scritti in Move.
- `move-core-types`: gestisce l'implementazione dei tipi di base in Move.
- `move-coverage`: implementa delle funzionalità per studiare la copertura del codice.
- `move-binary-format`: contiene la gestione del bytecode di Move e le funzioni per caricare in memoria i moduli compilati.
- `move-package`: contiene delle strutture dati utilizzare nella CLI per gestire i parametri di compilazione per i package di Move.
- `move-model`: contiene numerose funzioni per la lettura dei bytecode Move, utilizzate per localizzare la funzione target da eseguire e durante la generazione dei dati per ottenere in automatico il

tipo dei parametri delle funzioni. L'automazione di questo passaggio ha permesso di semplificare[2] di molto sia l'implementazione del fuzzer in sè che l'utilizzo del fuzzer da parte dell'utente.

- `move-stdlib-natives`: contiene la gestione delle funzioni native di Move. Le funzioni native sono funzioni della standard library di Move la cui implementazione è però scritta in Rust. Questo tipo di funzioni richiede una gestione separata durante la creazione della Move VM.

2.5 Configurazione iniziale

Come accennato nell'introduzione di questo capitolo, il progetto è composto da **CLI** e **move-libfuzzer**: due package differenti che, una volta installati, diventano due programmi eseguibili indipendenti. Sono dunque necessarie delle configurazioni iniziali per render possibile la comunicazione tra di essi.

Il primo requisito fondamentale è installare la toolchain di Rust per poter compilare il codice sorgente e installare gli eseguibili. Successivamente bisognerà installare la CLI di Move, la CLI del fuzzer ed infine il package *move-libfuzzer*. L'installazione avviene attraverso il comando *cargo install*, integrato nel package manager di Rust. I passaggi necessari sono integrati all'interno dello script *setup.sh*, scritto in Bash. Quest'ultimo installa in successione i tre eseguibili, impostando i flag necessari per la compilazione.

Parlando più nel dettaglio del processo di installazione, esso è composto dalle seguenti fasi:

1. La prima fase consiste nel riconoscimento della piattaforma di esecuzione, richiesta da *rustc* per la compilazione dei vari eseguibili. La piattaforma corrente può essere ottenuta tramite il seguente comando.

   ```
   CURRENT_PLATFORM=(rustc -vV | grep "host:" | sed 's/host: //')
   ```

   Questa istruzione può essere suddivisa in tre sotto-comandi: il primo chiama `rustc` (il compilatore di Rust) con l'opzione `-vV` (verbose-version) che fornisce informazioni dettagliate sulla sua versione tra cui l'*host* su cui viene eseguito. L'output dello scorso comando presenta tante righe diverse nel formato "<chiave>: <valore>". Tramite il secondo comando viene estratta la riga contenente le informazioni relative all'host utilizzando il comando *grep* mentre il terzo utilizza `sed`, uno stream-editor integrato in UNIX, per sostituire la stringa "host: " con una stringa vuota, ritornando soltanto il valore corrispondente alla piattaforma di esecuzione.

2. Il secondo passaggio controlla se la CLI di Move è già stata installata, per fare ciò si utilizza il seguente comando:

   ```
   cargo install --list | grep -q "move-cli"
   ```

   La prima parte ritorna l'elenco di tutti gli eseguibili installati tramite `cargo`, la seconda, utilizzando `grep`, cerca se l'eseguibile *move-cli* è presente nell'elenco. Se l'eseguibile viene trovato si prosegue con il setup, se invece non era già precedentemente installato si utilizza il *cargo-install* per compilarlo ed installarlo in una cartella predefinita di *cargo*.

   ```
   cargo install --path ./move-sui/crates/move-cli
   ```

   In questo caso l'installazione avviene utilizzando i parametri predefiniti.

3. Dopo aver installato la CLI di Move si prosegue con l'installazione della CLI di *move-fuzz*.

   ```
   cargo install --path ./cli --target $CURRENT_PLATFORM
   ```

[2] In una delle prime versioni, visto che il tipo dei parametri non veniva riconosciuto in automatico, era necessario implementare del codice in Rust per definire il tipo dei dati da generare e per impostare eventuali valori predefiniti. Il fatto di avere i target scritti in due linguaggi differenti complicava di molto il lavoro svolto dalla CLI e rendeva anche la struttura del progetto molto più confusionaria.

Anche in questo caso l'installazione avviene con i parametri predefiniti.

4. L'ultimo passaggio, il più complesso, riguarda installazione di *move-libfuzzer*. In questo è necessario impostare dei flag aggiuntivi utilizzando le variabili d'ambiente RUSTFLAGS e ASAN_OPTIONS.

   ```
   export RUSTFLAGS="
   -Cpasses=sancov-module \
   -Cllvm-args=-sanitizer-coverage-level=4 \
   -Cllvm-args=-sanitizer-coverage-inline-8bit-counters \
   -Cllvm-args=-sanitizer-coverage-pc-table \
   -Cllvm-args=-sanitizer-coverage-trace-compares \
   -Cdebug-assertions \
   -Ccodegen-units=1 \
   --cfg fuzzing \
   -Zsanitizer=address"

   export ASAN_OPTIONS="detect_odr_violation=0"
   ```

   Questi flag servono principalmente per abilitare e configurare il l'*address-sanitizer*: uno strumento utile per rilevare errori di memoria come buffer *overflow*, *use-after-free* e *memory leaks*. Dopo aver impostato tutti i flag necessari, viene eseguita l'installazione vera e propria, accertandosi di eseguirla in modalità *release*.

   ```
   cargo +nightly install --path ./move-fuzzer --target $CURRENT_PLATFORM
       --config profile.release.debug=true
   ```

Lo script *setup.sh* può opzionalmente accettare il parametro –clean che permette di chiamare il comando `cargo clean` sui package da installare prima di procedere ad una nuova installazione. Ciò permette di eliminare gli eventuali file di installazione esistenti in precedenza ed eseguire un l'installazione come se fosse la prima.

3 Package move-libfuzzer

Il package *move-libfuzzer*, come suggerisce il nome, ha come scopo quello di adattare al linguaggio Move le funzionalità di *libfuzzer*, un motore di fuzzing sviluppato in origine per i linguaggi C/C++. Nello specifico i principali compiti di *move-libfuzzer* sono:

- **caricare** i moduli Move da eseguire,
- **ricevere** i dati generati da *libfuzzer* in un formato grezzo (vettore di byte),
- **formattare** i dati come richiesto dalla Virtual Machine di Move,
- **eseguire** la funzione richiesta in accordo con i parametri ricevuti dalla CLI.

Le operazioni svolte dal package possono essere suddivise in due fasi distinte: **inizializzazione** ed **esecuzione**. Il caricamento dei moduli e la creazione della Virtual Machine avvengono una volta sola durante l'inizializzazione, tutti i passaggi che avvengono dalla generazione dei dati all'esecuzione effettiva della funzione target, vengono eseguiti ogni nuovo set di dati generato dal fuzzer.

Nel corso di questo capitolo verrà descritta in modo approfondita l'implementazione del package, motivando scelte di implementazione ed illustrando eventuali difficoltà riscontrate durante lo sviluppo. Il capitolo inizia trattando l'interazione con la libreria *libfuzzer* tramite FFI all'interno della sezione 3.1, successivamente verranno approfonditi i dettagli delle tre diverse fasi di esecuzione: **compilazione/linking** (sezione 3.1.1), **inizializzazione** (sezione 3.1.2) ed **esecuzione** (sezione 3.1.3). La sezione 3.2 infine è la più corposa del capitolo, riguarda il modulo *move-runner*: il componente principale di *move-libfuzzer*. La sua descrizione è divisa in numerose sotto-sezioni che trattano le diverse fasi dell'esecuzione del fuzzer, a partire dalla caricamento dei moduli fino all'esecuzione vera e propria delle funzioni. Viene inizialmente descritto il **caricamento** dei moduli nella sezione 3.2.1, viene successivamente aperta una parentesi riguardante la **gestione dei *data-types*** all'interno della sezione 3.2.2. Questa sezione è molto importante per la comprensione dei concetti espressi in seguito. La sezione 3.2.3 illustra come vengono analizzate le funzioni target per ricavare il tipo dei loro parametri mentre nella sezione 3.2.4 vengono descritte le ultime fasi dell'inizializzazione del runtime. Consiste nella creazione della Virtual Machine e dell'oggetto Move Runner. Le ultime sezioni riguardano la fase di esecuzione del fuzzer, in ordine le sezioni trattano: la **creazione della sessione** (sezione 3.2.6), la **generazione dei dati** (sezione 3.2.7) ed infine l'**esecuzione dei target** (sezione 3.2.8).

3.1 Interazione con libfuzzer tramite FFI

Come accennato nei capitoli precedenti, le funzioni vere e proprie relative al fuzzing risiedono all'interno del framework *libfuzzer*, scritto in C++. Il compito di *move-libfuzzer* è quello di creare un'interfaccia tra *libfuzzer* e il linguaggio Move implementando delle particolari funzioni tramite le FFI (Foreign Function Interface)[1]. È importante specificare che *libfuzzer* è il vero programma eseguibile di questo package e che per funzionare necessita dell'implementazione di due funzioni: "LLVMFuzzerInitialize" e "LLVMFuzzerTestOneInput". La prima viene chiamata in fase di inizializzazione del fuzzer mentre la seconda è il target vero e proprio del fuzzer: la funzione a cui verranno passati i dati del fuzzer come vettore di byte e dalla quale si svilupperà la maggior parte del codice di *move-libfuzzer*. In uno scenario normale, queste funzioni verrebbero implementate direttamente in C++. In questo caso, visto che l'implementazione di Move è scritta in Rust è necessaria una FFI per permettere la comunicazione tra

[1] Meccanismo che permette ad un programma scritto in un certo linguaggio di chiamare funzioni di un programma scritto in un altro linguaggio

i due linguaggi.

Per comprendere al meglio l'implementazione delle funzioni, è utile partire dalla definizione della FFI in C, all'interno di *libfuzzer*.

```
extern "C" {
    __attribute__((weak)) int LLVMFuzzerInitialize(int *argc, char ***argv);
    int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size);
}
```

Il blocco "`extern "c"`" viene utilizzato per definire una Foreign Function Interface (FFI), indica ad una funzione che deve seguire lo standard di C per quanto riguarda il *linking*. In C++ e in altri linguaggi, si utilizza una tecnica detta *name-mangling* per permettere l'overloading delle funzioni con lo stesso nome, consiste nel modificare i loro nomi includendo informazioni riguardo al loro *namespace* e ai loro parametri. Tuttavia, questo può rappresentare un problema quando lo sviluppatore vuole esporre tali funzioni ad altri programmi tramite una FFI, in questo caso il *linker* non sarebbe in grado di mappare le funzioni tra i due programmi visto che i nomi potrebbero non coincidere. Il linguaggio C, d'altro canto, non supporta l'overloading di funzioni con lo stesso nome, è quindi una tecnica comune quella di imporre al compilatore l'utilizzo dello standard di C per i nomi, disabilitando il *name-mangling* e rendendo possibile il lavoro del linker.

Un altro dettaglio interessante che riguarda la funzione `LLVMLibfuzzerInitialize` è che quest'ultima è stata dichiarata con l'attributo "__attribute__((weak))": una direttiva per il *linker* che permette di dichiarare la funzione come simbolo debole, il che rende facoltativa la sua l'implementazione. Il processo di *linking* può terminare senza errori anche se non viene trovata una sua implementazione forte, questa tecnica è spesso utilizzata per le funzioni di inizializzazione in quanto non sempre sono necessarie. È invece sempre richiesta l'implementazione della funzione LLVMFuzzerTestOneInput.

3.1.1 Compilazione e linking

La compilazione del crate *move-libfuzzer*, gestita da Cargo[2], richiede alcuni passaggi in più rispetto ad un progetto Rust classico. Questo è dovuto alla particolare struttura composta da codice scritto sia in Rust che in C++. Le problematiche principali da risolvere sono:

- la compilazione separata del codice in Rust e in C++;
- la gestione della fase di *linking* per creare un unico eseguile;
- la gestione del fatto che la funzione *main* non è definita in Rust ma si trova all'interno di libfuzzer.

La loro gestione è suddivisa nei file *Cargo.toml*, *build.rs*, *main.rs*. Il primo contiene tutte le configurazioni di Cargo, in particolare le configurazioni da aggiungere saranno:

- la creazione di un nuovo file binario collegato al file *main.rs*:

  ```
  [[bin]]
  name = "move-libfuzzer"
  path = "src/main.rs"
  ```

  Il file *main.rs* è un file sorgente Rust vuoto contenente soltanto l'istruzione "`#![no_main]`", questa direttiva configura il compilatore affinché ignori l'assenza della funzione *main* in quanto fornita da librerie esterne. In questo caso, come accennato in precedenza, la funzione *main* è definita all'interno di *libfuzzer*. Viene delegata quindi al linker il compito di ricercare la funzione `main` all'interno degli altri file binari, di includerla all'interno dell'eseguibile finale e di generare un errore in caso non venga trovata.

[2] Cargo è il package manager di Rust

- la definizione di una *feature* abilitata di default chiamata *link-libfuzzer*, permette di scegliere se attivare o disattivare la compilazione di *libfuzzer* all'interno dello script *build.rs*.

  ```
  [features]
  default = ["link_libfuzzer"]
  link_libfuzzer = []
  ```

- l'aggiuta delle dipendenze necessarie in fase di compilazione. CC è un crate che permette la compilazione di codice sorgente scritto in C.

  ```
  [build-dependencies]
  cc = { version = "1.0", features = ["parallel"] }
  ```

Il file *build.rs* è un file Rust che viene utilizzato per eseguire del codice prima che il package venga compilato da Cargo prima della compilazione di un package.

3.1.2 Inizializzazione

La fase di inizializzazione ha lo scopo di preparare l'ambiente per l'esecuzione delle funzioni target, i suoi compiti principali sono l'analisi della funzione da eseguire e il caricamento in memoria dei moduli necessari per la sua esecuzione. Questa procedura è gestita all'interno della funzione `initialize`, definita all'interno del file *lib.rs*. La sua esecuzione avviene a partire dall'interno di *libfuzzer*, perciò deve esportata con il nome e gli standard richiesti dalla FFI discussi in precedenza. Ciò avviene utilizzando la direttiva `#[export_name = "LLVMFuzzerInitialize"]` per impostare il nome con cui deve essere esportata e il comando `extern "c"` per imporre gli standard di compilazione di C ed evitare problemi in fase di linking.

```
#[export_name = "LLVMFuzzerInitialize"]
pub extern "C" fn initialize(
    _argc: *const isize,
    _argv: *const *const *const u8
) -> isize { ... }
```

La funzione accetta come parametri due valori:

- `_argc` è un puntatore ad un valore intero, rappresenta il numero di parametri ricevuti dalla riga di comando.
- `_argv` è un puntatore ad un vettore di stringhe, rappresenta i valori dei parametri ricevuti dalla riga di comando.

Questa funzione richiama la struttura classica dei programmi C, l'unica differenza è che in questo caso è stato aggiunto un livello ulteriore di puntatori per limitare l'utilizzo della memoria. Nonostante i parametri fossero disponibili attraverso i parametri della funzione, per la loro lettura è stata utilizzata la libreria *clap*. Quest'ultima offre un'interfaccia più semplice e intuitiva per la gestione di parametri complessi rispetto alla manipolazione diretta del vettore di stringhe. In questo modo si riduce anche il rischio associato all'uso di codice *unsafe* necessario per la manipolazione diretta dei puntatori in Rust.

La lettura degli argomenti a riga di comando avviene grazie alla struttura dati *cli*, definita come segue all'interno del file *lib.rs*:

```
#[derive(Clone, Debug, Eq, PartialEq, Parser)]
#[command(allow_hyphen_values = true)]
pub struct Cli {
    // command line arguments
}
```

La struct `Cli` deriva le funzionalità di `Parser`, il *trait*[3] utilizzato dalla libreria *Clap* per effettuare l'analisi della riga di comando valorizzando in automatico in campi della struttura dati utilizzata, questa operazione è svolta dalla funzione `parse()`, definita all'internod el trait stesso. I parametri da leggere riguardano le informazioni per il riconoscimento della funzione target in aggiunta ad alcune opzioni riguardanti la generazione dei dati sulla copertura. Bisogna inoltre preservare tutte le opzioni destinate a *libfuzzer*. Di seguito viene presentata la lista completa dei campi della struct `Cli`:

- `module_path:PathBuf`, rappresenta il percorso del modulo da analizzare. Il percorso deve puntate al *bytecode* già compilato del modulo contenente la funzione da testare.
- `target_module:String`, specifica il nome del modulo contenente la funzione da testare.
- `target_function:String`, indica il nome della funzione da testare.
- `coverage:bool`, indica se si desidera raccogliere dati di copertura durante l'esecuzione dei test.
- `coverage_map_dir:Option<PathBuf>`, opzionale. Se abilitato il coverage, specifica la directory dove salvare le mappe di copertura.
- `extra:Option<Vec<String>>`, opzionale. Consente di ignorare gli argomenti destinati a *libfuzzer* extra senza generare errori.

Grazie alle informazioni lette dalla riga di comando è possibile costruire l'oggetto `MOVE_RUNNER`. Memorizzato come variabile globale e conservato in maniera statica per tutta la durata dell'esecuzione.

```
let cli = Cli::parse();
MOVE_RUNNER.set(
    Mutex::new(
        MoveRunner::new(
            cli.module_path,
            &cli.target_module,
            &cli.target_function,
            cli.coverage,
            cli.coverage_map_dir,
        ),
    ),
).expect("Failed to initialize move runner");
```

Rust è un linguaggio che punta a garantire la sicurezza e la correttezza anche in un ambiente concorrente, di conseguenza la gestione delle variabili globali non è banale come in altri linguaggi. La variabile `MOVE_RUNNER` è stata dichiarata con il tipo `OnceCell<Mutex<MoveRunner>>`, ciò permette di definire un area di memoria in sola lettura, che può essere inizializzata una sola volta. Si utilizza invece un `Mutex` per permettere l'accesso concorrente ai dati da parte di più thread. L'utilizzo di `OnceCell` è importante quando si lavora con variabili globali perché garantisce che l'inizializzazione avvenga in modo controllato e senza la possibilità che il valore venga sovrascritto in futuro.

Il funzionamento dettagliato di *MoveRunner* verrà discusso più avanti nella lettura all'interno del capitolo 3.

3.1.3 Esecuzione

Durante la fase di esecuzione un test viene eseguito ripetutamente con dati diversi finché non viene trovato in input che causa un errore nel codice oppure finchè non si raggiunge un numero massimo di tentativi. Questa fase è gestita all'interno della funzione `test_input`, definita all'interno del file *lib.rs*. Anche questa funzione deve essere richiamata da *libfuzzer*, si utilizza perciò la stessa tecnica vista in precedenza durante l'inizializzazione per adattare la funzione alla FFI.

[3]Un trait in Rust è simile ad un'interfaccia in altri linguaggi di programmazione, permette di definire un comportamento comune per tipi di dati diversi. Sono più flessibili e possono anche includere metodi di default.

```
#[export_name = "LLVMFuzzerTestOneInput"]
pub fn test_input(data: *const u8, size: usize) -> i32 { ... }
```

La funzione accetta in input due argomenti:

- `data` è un vettore di byte (numeri interi da 8 bit), contiene i dati generati da *libfuzzer* e che dovranno essere passati alla funzione target.
- `size` è un valore intero che rappresenta la lunghezza del dei dati ricevuti.

Questi due valori, per essere utilizzati in maniera ottimale in Rust, devono essere convertiti in una variabile di tipo `&[u8]`, uno `slice`[4] contenente interi a 8 bit. L'accesso libero alla memoria e la manipolazione dei puntatori *raw* in Rust richiede l'utilizzo di codice *unsafe*, codice che normalmente non sarebbe consentito dal compilatore. È tuttavia possibile consentire l'utilizzo queste funzioni sotto la responsabilità del programmatore inserendole nel blocco un `unsafe`.

```
let data_slice : &[u8] = unsafe {
    std::slice::from_raw_parts(data, size)
};
```

La funzione `slice::from_raw_parts` costruisce uno `slice` a partire dal puntatore al primo elemento di un vettore e dal numero di elementi che esso contiene. Visto che la funzione accede direttamente alla memoria leggendo dei puntatori liberi, deve essere incapsulata all'interno di un blocco `unsafe` per essere utilizzata.

Una volta convertiti i dati in un formato più semplice da manipolare, la funzione `test_input` può proseguire con l'esecuzione della funzione target eseguendo la seguente porzione di codice.

```
let mut runner = MOVE_RUNNER.get().unwrap().lock().unwrap();
if let Err(e) = (*runner).execute(data_slice) {
    std::process::abort();
}
```

In questo blocco, si ottiene l'oggetto `MOVE_RUNNER` leggendo la variabile globale inizializzata precedentemente e acquisendo il *lock* sul mutex che la contiene. Viene successivamente eseguita la funzione `execute`. Essa contiene tutte le procedure necessarie per eseguire la funzione target, convertendo `data_slice` nei tipi dei parametri richiesti. Il suo contenuto verrà descritta in dettaglio nella sezione dedicata al modulo *move-runner*.

Se l'esecuzione della funzione `execute` fallisce, il processo viene terminato immediatamente con `std::process::abort()`. Facendo terminare il processo, si segnala a *libfuzzer* che si è verificato un errore durante l'esecuzione, innescando tutte le procedure per ottenere più informazioni possibili riguardo al fallimento. In questo modo, il fuzzer può raccogliere dettagli sull'errore, aiutando gli sviluppatori a identificare e risolvere eventuali errori e falle di sicurezza nel codice testato.

3.2 Modulo move-runner

Il modulo move-runner è il componente del progetto responsabile dell'esecuzione delle funzioni Move in tutti i loro aspetti. È suddiviso in diversi sotto-moduli, ognuno dei quali è incaricato di gestire uno specifico compito:

- `module_manager` gestisce il caricamento dei moduli Move necessari all'esecuzione e il loro salvataggio in memoria. La gestione dei queste operazione è gestita a sua volta in due diversi sotto-moduli: `module_loader` e `module_store`. Il primo viene utilizzato in fase di inizializzazione e permette di caricare in memoria il target-module e tutte le sue dipendenze. Il secondo definisce la struct `ModuleStore`, richiesta dalla sessione di Move per mantenere una mappa dei moduli a runtime.

[4] In Rust, uno slice è un riferimento che punta ad una porzione continua di un array o di un vettore, senza possederne i dati sottostanti. Consente di accedere a una sequenza di elementi contigui, specificando un intervallo di indici.

- `arbitrary_inputs` partendo dai dati ricevuti dal fuzzer come di vettore di byte, genera i dati da passare in input alle funzioni. Utilizzando i moduli caricati in memoria, viene letta la firma della funzione target in modo da generare gli input del tipo corretto.
- `types` si occupa della mappatura tra i tipi di Move e quelli riconosciuti dal fuzzer. Assicura che i tipi di dati utilizzati nelle funzioni Move siano correttamente interpretati e gestiti dal fuzzer.
- `utils` fornisce funzioni più generiche, utilizzate da tutti gli altri moduli.

L'approfondimento di *move-runner* verrà nuovamente suddiviso in fase di inizializzazione e fase di esecuzione.

3.2.1 Caricamento dei moduli - il modulo move_loader

La prima fase per l'esecuzione delle funzioni target è il caricamento in memoria dei moduli di Move, gestita da `module_loader` attraverso la creazione di un oggetto di tipo `ModuleLoader`, una struct che memorizza tre valori:

- `module_path:String`, il percorso al file contenente il bytecode del modulo compilato.
- `module:CompiledModule`, il modulo dopo che viene caricato,
- `dependencies:Vec<CompiledModule>`, un vettore contenente il bytecode di tutte le dipendenze del modulo "radice".

Durante la sua prima inizializzazione, che avviene all'interno della funzione `ModuleLoader::new()`, viene caricato in memoria il modulo "radice" e viene impostato come vettore vuoto il campo `dependencies`. Il caricamento delle dipendenze avviene invece all'interno della funzione `load_dependencies()`. Al suo interno si scorre l'albero delle cartelle a partire dal percorso del primo modulo e si caricano in memoria tutti i moduli Move che vengono trovati.

```
pub fn load_depencencies(&mut self) {
    for entry in WalkDir::new(self.get_root_dir())
        .into_iter()
        .filter_map(|e| e.ok()) {
        let path = entry.path();
        if path.is_file() && path != Path::new(self.module_path.as_str()) {
            if let Some(ext) = path.extension() {
                if ext == MOVE_COMPILED_EXTENSION {
                    self.dependencies.push(load_compiled_module(path.to_str().unwrap()));
                }
            }
        }
    }
}
```

All'interno della funzione `ModuleLoader::load_dependencies()` viene analizzato l'albero delle directory a partire dalla radice del progetto per caricare tutti i moduli. Questa operazione viene eseguita utilizzando l'oggetto `WalkDir`, fornito dall'omonima libreria, grazie al quale è possibile convertire l'albero di directory in un elenco di elementi iterabile con un classico ciclo `for`. In questa fase vengono selezionati soltanto i file con l'estensione *.mv*, l'estensione dei moduli compilati di move. Infine i moduli selezionati vengono caricati in memoria utilizzando la funzione `load_compiled_module()` per essere successivamente aggiunti al vettore delle dipendenze.

3.2.2 Gestione dei tipi - il modulo types

Per affrontare al meglio l'analisi delle funzioni di Move, è importante conoscere come è stata implementata la rappresentazione dei tipi di dato. All'interno di *move-runner* esistono infatti tre diverse enumerazioni, ognuna con scopi diversi:

- `MoveType`[5]: oggetti che contengono tutte le informazioni per rappresentare i tipi all'interno del

[5] Alias per il tipo `move_model::ty::Type`, definito all'interno del codice sorgente di Move.

bytecode di un modulo. È il tipo che viene ottenuto durante l'analisi delle funzioni.

- `MoveValue`: oggetti che rappresentano quali tipi possono essere utilizzati come argomenti di una funzione. È il tipo di dato richiesto dalla sessione di Move al momento di eseguire una funzione.

- `FuzzerType`: rappresenta i tipi che il fuzzer è in grado di interpretare, generare e quindi passare come argomento alle funzioni target. Viene utilizzata per semplificare la conversione tra `MoveType` e `MoveValue`.

Una differenza importante tra queste enumerazioni, che può essere intuita dal nome, è che `MoveType` e `FuzzerType` sono utilizzati durante l'analisi dei parametri, rappresentano soltanto il loro tipo senza il loro valore effettivo, `MoveValue` d'altro canto è utilizzato dalla sessione di Move per eseguire le funzione, perciò contiene anche le informazioni che riguardano il contenuto effettivo della variabile.

Nella tabella 3.1 sono riassunti gli elementi che appartengono alle tre enumerazioni menzionate in precedenza, si può notare che `MoveType` contiene molti tipi in più rispetto alle altre due, sono infatti presenti anche tipi speciali che non possono essere utilizzati come parametri di funzioni. Fanno parte di questo elenco funzioni, *type-parameters*, errori, riferimenti ecc... `FuzzerType` e `MoveValue`, invece, contengono gli stessi tipi, ciò significa che il fuzzer è in grado di generare tutti i tipi che le funzioni di Move possono accettare.

Tipo	**MoveType**	**FuzzerType**	**MoveValue**
Bool	✓	✓	✓
U8	✓	✓	✓
U16	✓	✓	✓
U32	✓	✓	✓
U64	✓	✓	✓
U128	✓	✓	✓
U256	✓	✓	✓
Address	✓	✓	✓
Signer	✓	✓	✓
Tuple	✓		
Vector	✓	✓	✓
Struct	✓	✓	✓
TypeParameter	✓		
Reference	✓		
Fun	✓		
TypeDomain	✓		
ResourceDomain	✓		
Error	✓		
Var	✓		
Num	✓		
Range	✓		
EventStore	✓		

Tabella 3.1: Comparazione dei tipi presenti in MoveType, FuzzerType e MoveValue

La conversione `MoveType` a `FuzzerType` avviene con la funzione `FuzzerType::from(env:&GlobalEnv, value:MoveType)`, al suo interno avviene il mapping dei tipi primitivi, dei vettori e delle struct. I tipi primitivi vengono convertiti in maniera diretta e non necessitano particolari elaborazioni, vettori e struct necessitano invece di alcuni passaggi aggiuntivi descritti nel prossimo paragrafo.

	Primitive	Vector	Struct
MoveType	U32	Vector(Box<MoveType>)	Struct(ModuleId, StructId, Vec<Type>)
FuzzerType	U32	Vector(Box<FuzzerType>)	Struct(Vec<FuzzerType>)
Move Value	U32(u32)	Vector(Vec<MoveValue>)	Struct(MoveStruct)[6]

Tabella 3.2: Implementazione dei tipi nelle diverse enumerazioni

Rappresentazione di vettori e struct

Vettori, struct e tipi primitivi sono rappresentati all'interno del fuzzer attraverso le enumerazioni elencate in precedenza, nella tabella 3.2 vengono messe a confronto le implementazioni di ogni tipo nelle tre enumerazioni.

Come è possibile notare, l'implementazione dei tipi primitivi consiste in un classico elemento della enumerazione (è stato usato `U32` come esempio ma tutti gli altri tipi sono implementati allo stesso modo). Struct e vettori richiedono invece dei parametri aggiuntivi. Le struct vengono rappresentate all'interno del fuzzer utilizzando un vettore di MoveType, le strutture dati possono infatti contenere campi di tipi diversi, I vettori sono invece rappresentati soltanto attraverso il tipo dei propri elementi, visto che sono tutti dello stesso tipo.

Andiamo ora a trattare la conversione vera e propria tra MoveType e FuzzerType che per quanto riguarda i vettori, avviene in modo ricorsivo con il codice seguente:

```
pub fn from(env: &GlobalEnv, move_type: MoveType) -> Self {
    match move_type {
        ...
        MoveType::Vector(inner_type: Box<MoveType>) => {
            FuzzerType::Vector(Box::new(FuzzerType::from(env, *inner_type)))
        },
        ...
    }
}
```

Le struct, d'altro canto, richiedono alcuni passaggi aggiuntivi rispetto ai vettori, sono infatti memorizzate all'interno di `MoveType`, attraverso il proprio ID (`struct_id`) e l'ID del modulo che le contiene (`module_id`). Sono perciò necessari dei passaggi aggiuntivi per ottenere l'elenco dei campi tramite il `GlobalEnv`[7]. Il terzo parametro, di tipo `Vec<MoveType>`, rappresenta i *type-parameters* di quella struct. Non sono tuttavia necessari in questa versione del fuzzer in quanto non viene supportata la possibilità di generare tipi generici attraverso il fuzzer. Per ottenere i campi di una struct è stato utilizzato il codice riportato in seguito:

```
pub fn from(env: &GlobalEnv, move_type: MoveType) -> Self {
    match move_type {
        ...
        MoveType::Struct(module_id, struct_id, _) => {
            let module_env = env.get_modules().find(|m| m.get_id() ==   module_id).unwrap();
            let struct_env = module_env.get_struct(struct_id);
            let fields = struct_env
                .get_fields().map(|f| f.get_type())
                .collect::<Vec<MoveType>>();
            FuzzerType::Struct(fields.into_iter()
                .map(|t| FuzzerType::from(env, t))
                .collect_vec())
```

[7] `GlobalEnv` è la struttura dati utilizzata da Move per rappresentare l'ambiente globale con tutti i moduli caricati in memoria.

```
        }
        ...
    }
}
```

All'interno della funzione si accede al Global Environment per ottenere in ordine l'ambiente specifico di del modulo in questione (`module_env`), l'ambiente specifico della struct (`struct_env` ed infine il tipo dei suoi campi. Questi ultimi vengono infine convertiti dall'enumerazione `MoveType` a `FuzzerType`, così da poter essere utilizzati nelle fasi successive.

La conversione da `FuzzerType` a `MoveValue` avviene durante la generazione dei dati da passare alle funzioni, verrà approfondita in seguito nel capitolo riguardante il modulo `arbitrary_inputs` (sezione 3.2.7).

3.2.3 Analisi della funzione target

L'analisi delle funzioni target viene eseguita durante l'inizializzazione del fuzzer ed è la fase che segue il caricamento dei moduli. Durante questa fase vengono analizzati i moduli di Move per ricavare le funzioni target e quindi la loro firma. Ciò avviene in maniera molto simile a quella utilizzata in precedenza per l'analisi delle struct.

Il processo inizia con la creazione dell'oggetto `GlobalEnv`, utilizzato per ricavare in ordine `ModuleEnv` e `FunctionEnv`, necessari successivamente per l'analisi della funzione. Il processo avviene con i seguenti passaggi:

- creazione dell'oggetto *module_map* a partire dai moduli caricati in precedenza durante l'inizializzazione.
- calcolo del grafo delle dipendenze con la funzione `compute_dependency_graph()`,
- ordinamento topologico del grafo delle dipendenze,
- creazione dell'ambiente globale e aggiunta di tutti i moduli al suo interno seguendo l'ordine topologico. È fondamentale rispettare l'ordine topologico per fare in modo che tutte le dipendenze siano rispettate.
- ricerca del modulo target all'interno del *global-env* per ottenere l'ambiente specifico di quel modulo, fondamentale per l'analisi delle funzioni.

```
pub fn get_function_parameters(
        modules: Vec<CompiledModule>,
        module_name: &str,
        function_name: &str) -> Vec<FuzzerType> {
    let params;
    let module_map = Modules::new(modules.iter());
    let dep_graph = module_map.compute_dependency_graph();
    let topological_order = dep_graph.compute_topological_order().unwrap();

    let mut env = GlobalEnv::new();
    add_modules_to_model(&mut env, topological_order);

    let module_env = env.get_modules().find(|m| m.matches_name(module_name));
    ...
}
```

Il passaggio successivo consiste nella ricerca della funzione target all'interno dell'ambiente suo modulo. L'operazione è svolta chiamando il metodo `get_functions()` della struct `ModuleEnv` ed impostando un filtro sul nome per ottenere soltanto la funzione target.

```
if let Some(env) = module_env {
    let function_env = env.get_functions()
                        .find(|f| f.get_name_str() == function_name);
    ...
}
```

L'ultimo passaggio dell'analisi consiste nel ricavare i tipi dei parametri della funzione utilizzando il metodo `func.get_parameter_types()` che ritorna un `Vec<MoveType>`. Utilizzando la funzione `transform_params`, si convertono i parametri da `MoveType` a `FuzzerType`.

```
if let Some(f) = func {
    params = f.get_parameter_types();
    transform_params(&env, params)
}
```

3.2.4 Creazione della MoveVM e di MoveRunner

L'ultimo passaggio che fa parte dell'inizializzazione di `move-libfuzzer` è la creazione della Virtual Machine di Move seguita dalla creazione dell'oggetto MoveRunner. La creazione della Move Virtual Machine avviene con la seguente porzione di codice:

```
let mut natives = all_natives(AccountAddress::ONE,
                                GasParameters::zeros());
let mut nursery_natives = nursery_natives(true, AccountAddress::ONE,
                                NurseryGasParameters::zeros());
natives.append(&mut nursery_natives);
let move_vm = MoveVM::new_with_config(natives, VMConfig::default()).unwrap();
```

Nella parte iniziale vengono caricate le *native-function* di Move: delle funzioni all'interno della standard library di Move la cui implementazione è scritta in Rust. Le funzioni di questo tipo non vengono caricate insieme ai moduli classici ma richiedono un procedimento separato. Una volta ottenute le funzioni native è possibile istanziare la Move Virtual Machine. In questo caso è stata creata con i parametri di default.

Il passaggio finale dell'inizializzazione del fuzzer è l'effettiva creazione dell'oggetto MoveRunner che contiene le seguenti proprietà:

- `move_vm:MoveVM`: l'oggetto che rappresenta la *virtual machine* di Move creato nei passaggi precedenti. Sarà necessario durante l'esecuzione per creare la sessione.
- `module:CompiledModule`: il bytecode modulo radice compilato
- `dependencies:Vec<CompiledModule>`: un vettore contenente il bytecode compilato delle dipendenze.
- `target_module:String`: il nome del modulo contenente la funzione target;
- `target_function:TargetFunction`: oggetto che rappresenta la funzione target. Contiene sia il suo nome che l'elenco dei suoi parametri.
- `coverage:bool`: un parametro ricevuto dalla CLI che indica se attivare oppure no la generazione dei report sulla copertura del codice.
- `coverage_map_dir:Option<PathBuf>`: un parametro ricevuto dalla CLI che indica il percorso del file dove memorizzare la *coverage-map*, il file prodotto dall'analisi della copertura.

3.2.5 Gestione dei moduli durante il runtime

Una volta conclusa la fase di inizializzazione, che viene effettuata una volta sola all'inizio dell'esecuzione del fuzzer, si procede con la fase di esecuzione. Questa seconda fase verrà eseguita molteplici volte, una per ogni nuovo set di dati generato dal fuzzer finché non viene trovato un errore o finché non si raggiunge un numero massimo di tentativi.

In questa fase i moduli precedentemente caricati durante l'inizializzazione vengono importati in una struttura dati detta `ModuleStore`, ciò permette alla sessione di Move di poterli utilizzare quando necessario. All'interno di Move, un `ModuleStore` è una qualsiasi struttura dati che implementa i trait `LinkageResolver`, `ModuleResolver`, `ResourceResolver`.

Il trait `LinkageResolver` viene utilizzato qualora il software necessitasse di rimappare i moduli a cui si fa riferimento, consentendo di eseguire lo stesso modulo all'interno dello *storage* con diverse dipendenze. All'interno di *move-fuzz* è stata mantenuta l'implementazione predefinita, la quale non esegue alcuna rilocazione.

La funzionalità di `ModuleResolver` è stata invece implementata come segue:

```
pub struct ModuleStore {
    modules: HashMap<ModuleId, Vec<u8>>,
}

impl ModuleResolver for ModuleStore {
    type Error = VMError;
    fn get_module(&self, module_id: &ModuleId) -> Result<Option<Vec<u8>>, Self::Error> {
        Ok(self.modules.get(module_id).cloned())
    }
}
```

Lo scopo di questa funzione è quello di indicare alla sessione di Move come ricavare un modulo dato il proprio ID. In questo caso i moduli sono memorizzati all'interno di un HashMap dove la chiave è l'id del modulo e il valore è un vettore di byte (`u8`) che memorizza il contenuto del modulo. Per ottenere un modulo dato il suo ID si può utilizzare la funzione `get` per accedere al contenuto della HashMap in corrispondenza di una certa chiave.

Per quanto riguarda `ResourceResolver`, è una funzionalità appartenente alla versione di Move nativo che non viene però più supportata all'interno di *move-sui*. È invece stata quindi scritta un'implementazione in maniera molto basilare utilizzata solo come segnaposto. Viene ritornato dalla funzione `Ok(None)` che significa che l'esecuzione non ha prodotto errori e che nessuna risorsa è stata trovata.

3.2.6 Creazione della sessione

Il primo passaggio dell'esecuzione di una funzione target, dopo il caricamento dei moduli all'interno di `ModuleStore`, è la creazione di una sessione. La sessione è l'oggetto che permette di creare un contesto di esecuzione per le funzioni da testare. Una sessione si ricava a partire dalla Move Virtual Machine, utilizzando il metodo `new_session`. Quest'ultimo prende in ingresso l'oggetto di tipo ModuleStore creato in precedenza e ritorna una nuova sessione di Move.

```
let mut store = ModuleStore::new(self.module.clone());
store.add_dependencies(&self.dependencies);
let mut session = self.move_vm.new_session(&store);
```

3.2.7 Generazione degli input - il modulo arbitrary_inputs

La generazione degli input è l'ultima fase che precede l'esecuzione della funzione target. Consiste nel trasformare i dati ricevuti da `libfuzzer` sotto forma di vettore di byte grezzo, nei tipi di dato richiesti

dai parametri della funzione target. Questa operazione avviene utilizzando la libreria `Arbitrary`, utilizzata anche dal fuzzer di Rust. In Rust la generazione dei dati avviene grazie al *trait* `Arbitrary`, implementato dall'omonima libreria per la maggior parte dei tipi presenti nella standard library di Rust e consiste nella generazione di un particolare tipo di dato a partire da dei bytes grezzi. Grazie alle caratteristiche di Rust, se una struttura dati è formata da attributi che implementano a loro volta `Arbitrary`, la struttura stessa "eredita" l'implementazione, se invece un tipo richiede una conversione personalizzata è possibile implementare la propria versione della funzione `arbitrary`. Nel fuzzer per Move invece, essendo un linguaggio molto più semplice di Rust, questo processo è stato automatizzato e nascosto agli occhi dell'utente. Durante la fase di inizializzazione vengono analizzate le funzioni target per ottenere il tipo dei loro parametri, così da poterne generare il contenuto in fase di runtime.

```
fn get_target_parameters(&self) -> Vec<FuzzerType> {
    self.target_function.args.clone()
}

pub fn execute(&mut self, bytes: &[u8]) -> Result<(), Error> {
    ...
    let input_types : Vec<FuzzerType> = self.get_target_parameters();
    let mut data = Unstructured::new(bytes);
    let arbitrary_data : Vec<MoveValue> = arbitrary_inputs(
                                            input_types.clone(),
                                            &mut data);
    ...
}
```

I tipi dei parametri della funzione target, ricavati in fase di inizializzazione, sono memorizzati all'interno dell'oggetto `MoveRunner`. Si possono leggere con la funzione `MoveRunner::get_target_parameters`. Vengono poi convertiti in un vettore di `MoveValue` attraverso la funzione `arbitrary_inputs`. L'oggetto di tipo `Unstructured` è implementato all'interno della libreria `Arbitrary` e permette di svolgere operazioni più elaborate su de bytes grezzi, come generare tipi di dato specifici partendo dai singoli bytes.

```
pub fn arbitrary_inputs(inputs: Vec<FuzzerType>,
        data: &mut arbitrary::Unstructured) -> Vec<MoveValue> {
    let mut res = vec![];
    for input in inputs {
        let value = arbitrary_input(input, data)
            .expect("Arbitrary error")
            .expect("Parsing error");
        res.push(value);
    }
    println!("{:?}", res);
    res
}
```

La funzione `arbitrary_inputs` prende in ingresso un vettore di `FuzzerType`, rappresentante l'elenco dei parametri da generare. Richiede inoltre un oggetto di tipo `Unstructured` che, come accennato in precedenza, contiene i dati generati dal fuzzer. La funzione scorre il vettore degli inputs e chiama su ogni elemento una funzione chiamata `arbitrary_input` (al singolare). Questa funzione si occupa di generare un oggetto di tipo `MoveValue` con i dati effettivi al suo interno. Questo procedimento avviene in maniera ricorsiva: i tipi primitivi sono i casi base, sono i tipi che il fuzzer è effettivamente in grado di generare, vi sono dei passi ricorsivi quando si incontrano vettori o struct. La loro gestione verrà approfondita in seguito.

```
fn arbitrary_input(input: FuzzerType, data: &mut arbitrary::Unstructured) ->
        ArbitraryResult<Result<MoveValue, Error>> {
    match input {
FuzzerType::Bool => Ok(Ok(MoveValue::Bool(<bool as Arbitrary>::arbitrary(data)?))),
FuzzerType::U8 => Ok(Ok(MoveValue::U8(<u8 as Arbitrary>::arbitrary(data)?))),
FuzzerType::U16 => Ok(Ok(MoveValue::U16(<u16 as Arbitrary>::arbitrary(data)?))),
FuzzerType::U32 => Ok(Ok(MoveValue::U32(<u32 as Arbitrary>::arbitrary(data)?))),
FuzzerType::U64 => Ok(Ok(MoveValue::U64(<u64 as Arbitrary>::arbitrary(data)?))),
FuzzerType::U128 => Ok(Ok(MoveValue::U128(<u128 as Arbitrary>::arbitrary(data)?))),
FuzzerType::U256 => Ok(Ok(MoveValue::U256(arbitrary_u256(data)?))),
FuzzerType::Vector(t) => Ok(arbitrary_vec(data, *t)?),
FuzzerType::Struct(values) =>
    Ok(Ok(MoveValue::Struct(MoveStruct(arbitrary_inputs(values, data))))),
FuzzerType::Address => Ok(arbitrary_address(data)?),
FuzzerType::Signer => Ok(arbitrary_signer(data)?),
    }
}
```

Per quanto riguarda i tipi primitivi, interi e booleani, la generazione avviene in maniera semplice utilizzando la libreria *arbitrary*. Per vettori, struct, *address* e *signer* sono richiesti passaggi differenziati.

L'implementazione per le struct prevede di chiamare in maniera ricorsiva la funzione `arbitrary_inputs` passando l'elenco dei propri campi come argomento. La ricorsione si ferma quando viene trovato un tipo primitivo.

```
fn arbitrary_account(u: &mut Unstructured) ->
ArbitraryResult<Result<AccountAddress, AccountAddressParseError>> {
    let mut buf = [0; mem::size_of::<AccountAddress>()];
    u.fill_buffer(&mut buf)?;
    Ok(AccountAddress::from_bytes(&buf))
}

fn arbitrary_address(u: &mut Unstructured) ->
ArbitraryResult<Result<MoveValue, Error>> {
    let res = match arbitrary_account(u)? {
        Ok(account) => Ok(MoveValue::Address(account)),
        Err(e) => Err(Error::AccountAddressParseError { message: e.to_string() }),
    };
    Ok(res)
}

fn arbitrary_signer(u: &mut Unstructured) ->
ArbitraryResult<Result<MoveValue, Error>> {
    let res = match arbitrary_account(u)? {
        Ok(account) => Ok(MoveValue::Signer(account)),
        Err(e) => Err(Error::AccountAddressParseError { message: e.to_string() }),
    };
    Ok(res)
}
```

L'implementazione di `Address` e `Signer` è contenuta rispettivamente nelle funzioni `arbitrary_address` e `arbitrary_signer`, implementate come riportato in precedenza. Il codice delle due funzioni è molto simile in quanto entrambi i tipi nascondono al loro interno un oggetto di tipo `AccountAddress`, gene-

rato a partire da un buffer di dati all'interno della funzione `arbitrary_account`.

I tipi trattati fin ora hanno tutti una dimensione predefinita, è semplice quindi stimare in partenza il numero di bytes da allocare per la loro generazione. I vettori, d'altro canto, non hanno una dimensione predefinita, serve quindi un modo per generare vettori di qualsiasi dimensione in base alla grandezza del buffer di dati. La generazione dei vettori è stata quindi implementata a partire da una struttura dati chiamata `ArbitraryIter`, si tratta di un iteratore che, invece di scorrere su un vettore già esistente, genera nuovi elementi finché ci sono bytes disponibili nel buffer.

```
struct ArbitraryIter<'a, 'b> {
    u: &'b mut Unstructured<'a>,
    t: FuzzerType
}

impl<'a, 'b> Iterator for ArbitraryIter<'a, 'b> {
    type Item = MoveValue;
    fn next(&mut self) -> Option<MoveValue> {
        let keep_going = self.u.arbitrary().unwrap_or(false);
        if keep_going {
            Some(arbitrary_input(self.t.clone(), self.u).unwrap().unwrap())
        } else {
            None
        }
    }
}
```

All'interno della funzione `next`, il fuzzer prova a generare nuovi dati attraverso la funzione `arbitrary`: se il procedimento va a buon fine si procede generando un nuovo elemento del vettore, altrimenti la generazione si ferma. Grazie all'implementazione di questo iteratore è possibile generare un vettore di una lunghezza variabile in base alla quantità di dati generati dal fuzzer.

```
fn arbitrary_iter<'a, 'b>(
    u: &'b mut Unstructured<'a>, fuzzer_type: FuzzerType
) -> ArbitraryResult<ArbitraryIter<'a, 'b>> {
    Ok(ArbitraryIter {u, t: fuzzer_type})
}

fn arbitrary_vec<'a, 'b>(
    u: &'b mut Unstructured<'a>, fuzzer_type: FuzzerType
) -> ArbitraryResult<Result<MoveValue, Error>> {
    Ok(Ok(MoveValue::Vector(
        arbitrary_iter(u, fuzzer_type)?.collect()))
    )
}
```

All'interno del codice della funzione `arbitrary_vec`, si sfrutta il metodo `collect` degli iteratori di Rust. Permette di trasformare un iteratore in un vettore vero e proprio scorrendo i vari elementi attraverso la funzione `next`. In questo caso la funzione next genera nuovi elementi fino ad esaurire il buffer di dati a sua disposizione, riuscendo quindi a generare un vettore di lunghezza variabile.

3.2.8 Esecuzione dei target

L'ultima fase dell'esecuzione del fuzzer è l'avvio effettivo della funzione target. L'avvio avviene grazie al metodo `session.execute_function_bypass_visibility`, quest'ultima accetta molteplici parametri:

- `module:&ModuleId`: l'ID del modulo contenente la funzione target;
- `function_name:&IdentStr`: il nome della funzione target;
- `ty_args:Vec<Type>` : i *type-parameters* della funzione target, non attualmente supportati dal fuzzer.
- `args:Vec<impl Borrow<[u8]>>` : i parametri della funzione serializzati in dei vettori statici di byte;
- `gas_meter:&mut impl GasMeter`: il gas è una metrica utilizzata da Move per tenere traccia del consumo delle risorse durante le transazioni. Al momento il fuzzer non ne tiene conto ma in futuro potrebbe essere supportato.

Di seguito viene riportato il codice utilizzato per l'esecuzione delle funzioni target: si può notare che vengono in ordine passati l'ID del modulo, il nome della funzione target (castato ad `IdentStr`), un vettore vuoto che rappresenta i *type-parameters* della funzione, i dati da passare alla funzione come parametri, serializzati in vettori di bytes utilizzando la funzione `serialize_values`, ed infine l'oggetto `UnmeteredGasMeter` per disabilitare i limiti sull'utilizzo delle risorse durante il fuzzing.

```
let result = session.execute_function_bypass_visibility(
    &self.module.self_id(),
    IdentStr::new(&self.target_function.name).unwrap(),
    vec![],
    serialize_values(&arbitrary_data),
    &mut UnmeteredGasMeter
);
```

Terminata l'esecuzione viene controllato lo stato dell'esecuzione e, se ci sono stati degli errori, vengono riportati prima di terminare l'esecuzione del fuzzer.

4 Package CLI

Fino ad ora è stata approfondito il package *move-libfuzzer*, quello più a stretto contatto con le funzionalità del fuzzer e con il runtime di Move. Per far sì che possa funzionare al meglio, serve che l'ambiente d'esecuzione sia configurato correttamente. Il package CLI (Command Line Interface) ha proprio il compito di configurarlo seguendo le richieste dell'utente e gestendo le logiche per eseguire al meglio *move-libfuzzer*. Per l'implementazione della CLI è stata utilizzata una libreria di Rust chiamata *Clap*, la quale rende molto più semplice la lettura degli argomenti da riga di comando ed automatizza molti aspetti di una CLI come i comandi `help`. Questa libreria permette infatti di trattare gli argomenti a riga di comandi come degli oggetti che vengono valorizzati in automatico dalla libreria. Grazie agli attributi dei campi delle struct è inoltre possibile imporre valori di default o dipendenze tra argomenti.

All'interno di questo capitolo verranno descritti i dettagli di implementazione della CLI, verrà inizialmente illustrata la fase di inizializzazione della CLI (sezione 4.1), una parte di codice in comune a tutti i comandi. Verranno successivamente descritti tutti gli altri comandi singolarmente. I comandi principali sono:

- `init`: inizializzare il progetto di fuzzing (sezione 4.2;
- `add`: aggiunge un nuovo target (sezione 4.3);
- `build`: compila il progetto di fuzzing (sezione 4.4);
- `run`: esegue il fuzzer su un determinato target (sezione 4.5).
- `fmt`: esegue una sola volta un determinato target passando un input specifico (sezione 4.6).
- `list`: mostra a schermo l'elenco dei target presenti nel progetto.

Sono stati implementati altri comandi relativi ad altre funzioni secondarie del fuzzer come la generazione di un report sulla copertura del codice, la minimizzazione del corpus e dei test case. Queste opzioni verranno approfondite all'interno della sezione 4.7 .

4.1 Inizializzazione della CLI

Tutte le funzionalità della CLI sono gestite all'interno di una struttura dati chiamata `Project`. Quest'ultima tiene traccia di quale sia la directory del progetto di fuzzing (campo `fuzz_dir`) e di quali siano i target (campo `targets`). L'esecuzione inizia ricercando il file `Move.toml` del progetto da testare; questo sarà il punto di partenza per la creazione delle *sub-directory* necessarie. È possibile specificare una `fuzz-dir` differente da quella predefinita utilizzando il parametro `--fuzz-dir`. Il secondo passaggio è il riconoscimento dei target già esistenti. Viene letto il contenuto della cartella `<fuzz-dir>/fuzz/sources`, la quale contiene un file diverso per ogni target.

```
|-- fuzz/
|   |-- sources/
|   |   |-- fuzz_target_1.move
|   |   |-- fuzz_target_2.move
|   |-- Move.toml
```

Prendendo come esempio l'albero di cartelle riportato in precedenza, sono presenti due target: `fuzz_target_1` e `fuzz_target_2`.

4.2 Il comando INIT

Il comando `init` è il primo comando che l'utente deve eseguire per preparare un nuovo progetto per il fuzzing. Nello specifico questo comando effettua i seguenti passaggi:

- crea una nuova directory denominata `fuzz` a partire dalla *fuzz-dir*;
- crea il file `Move.toml` del nuovo progetto Move dedicato al fuzzer. All'interno del progetto viene importata in automatico la *standard library* di Move.
- crea un file `.gitignore` per dire a Git di ignorare le cartelle contenenti i dati dal fuzzer durante il runtime, contenuti all'interno delle cartelle *target*, *corpus*, *artifacts* e *coverage*.
- crea il file `fuzz_target_1.move` all'interno della cartella `fuzz/sources/`, contenente il primo target del fuzzer. La creazione avviene utilizzando il seguente pattern:

```
module fuzz::{target_name} {
    public fun fuzz_target(bytes: vector<u8>) {
        // fuzz logic here
    }
}
```

L'utente può scegliere opzionalmente un nome diverso per il primo target da creare utilizzando l'argomento `--target`. La struttura finale che avrà il progetto una volta lanciato il comando init sarà:

```
dummy-project/
|-- fuzz/
|   |-- sources/
|       |-- fuzz_target_1.move
|   |-- .gitignore
|   |-- Move.toml
|-- sources/
|   |-- dummy_code.move
|-- Move.toml
```

4.3 Il comando ADD

Il comando `add` aggiunge un nuovo target al fuzzer. Il comando funziona in maniera simile ad `init` per quanto riguarda la creazione dei target, all'interno di questo comando viene saltata la logica per l'inizializzazione dell'ambiente di esecuzione.

4.4 Il comando BUILD

Il comando `build` è uno dei comandi più importanti di *move-fuzz*, permette di compilare il progetto di fuzzing, apportando tutte le opportune configurazioni. Visto che la compilazione dei moduli di Move è una fase comune a tutti i comandi che verranno trattati in seguito, gli argomenti accettati da questo comando saranno analoghi anche nei prossimi comandi.

4.4.1 Selezione del target

Una delle informazioni più importanti per qualsiasi comando del fuzzer è la funzione target sulla quale deve essere eseguito. È possibile indicare le funzioni target in due modi: attraverso il proprio nome e il nome del modulo al quale appartengono, se sono funzioni qualsiasi all'interno del codice, oppure tramite il nome del target se sono funzioni generate tramite la CLI. Nello specifico i parametri per indicare un target sono:

- `--target-module` e `--target-function`: utilizzati in coppia permettono di identificare una qualsiasi funzione all'interno di un progetto Move.

- `--target-name`: permette di identificare uno dei target specifici per il fuzzer, questi ultimi hanno infatti una struttura predefinita: il nome del modulo varia in base al nome del target, il nome della funzione è invece costante: tutte le funzioni si hanno il nome di `fuzz_target`. Ciò permette all'utente di utilizzare un solo parametro per richiamare i propri target, risultando in dei comandi più compatti.

Facendo riferimento all'albero di directory e al codice mostrato all'interno della sezione 4.2, ci sono due modi per identificare la funzione target in questione:

- `move-fuzz <cmd> --target-module fuzz_target_1 --target-function fuzz_target`
- `move-fuzz <cmd> --target-name fuzz_target_1`

Entrambi i metodi sono corretti, il secondo è tuttavia più compatto e semplice da utilizzare.

4.4.2 Configurazioni del compilatore

Per quanto riguarda gli altri argomenti che accetta il comando `build`, sono tutti parametri che accetta il comando originale del compilatore di Move. È possibile consultare la lista completa utilizzando l'opzione `--help` del comando `move build` o del comando `move-fuzz build --help`.

4.5 Il comando RUN

Il comando `run` ha il compito di eseguire il fuzzer su un determinato target offrendo numerose possibilità di configurazione. Si occupa di compilare i moduli Move, di effettuare eventuali configurazioni all'ambiente di esecuzione ed infine esegue *move-libfuzzer* secondo i parametri impostati dall'utente. Le opzioni che si possono configurare riguardano principalmente le impostazioni del compilatore e di *libfuzzer*. Le prime già sono state approfondite nella sezione precedente, per quanto riguarda *libfuzzer* si possono invece impostare numerosi parametri, consultabili dalla documentazione ufficiale. Le più importanti sono:

- `-max_len=<len>`‘: limita la lunghezza della stringa di input a ‘`<len>`‘.
- `-runs=<number>`‘: limita il numero di tentativi che il fuzzer effettua prima di terminare.
- `-max_total_time=<time>`‘: limita il tempo di esecuzione totale del fuzzer (in secondi).
- `-timeout=<time>`‘: imposta il tempo di esecuzione massimo per un singolo tentativo (in secondi), dopo il quale l'esecuzione verrà considerata fallita.

Come accennato in precedenza, la prima operazione svolta dal comando `run`, è la compilazione del codice Move, quest'ultima avviene in modo analogo a quanto illustrato per il comando `build`.

Le fasi successive riguardano la costruzione del comando per eseguire *move-libfuzzer*, il quale richiede diverse informazioni per il caricamento della funzione target, oltre ad alcuni parametri riguardanti il coverage e altri aspetti del fuzzer che verranno approfonditi in seguito nella sezione 4.7:

- `module_path`: il percorso al file compilato contenente il *target-module*.
- `target_module`: il nome del *target-module*.
- `target_function`: il nome della funzione target da eseguire.

Come è possibile notare, *move-libfuzzer* identifica una funzione tramite il proprio nome e il proprio modulo, non è a conoscenza della logica riguardante i *fuzz-target* i quali esistono soltanto a livello della CLI. Il compito del comando `run`, così come altri comandi che utilizzano *move-libfuzzer*, è quello di fornire questi tre parametri astraendo la logica dei target. Per fare ciò è stata utilizzata la struttura dati `Target` implementata come segue:

```
pub struct Target {
    pub target_module: Option<String>,
    pub target_function: Option<String>,
    pub target_name: Option<String>,
}

const DEFAULT_FUZZ_FUNCTION_NAME : &str = "fuzz_target";

pub fn get_target_module(&self) -> String {
    if let Some (module) = self.target_module.clone() { module }
    else { self.target_name.clone().expect("...") }
}

pub fn get_target_function(&self) -> String {
    if let Some (fun) = self.target_function.clone() { fun }
    else { String::from(DEFAULT_FUZZ_FUNCTION_NAME) }
}
```

La struct contiene tre campi: `target_name`, `target_module` e `target_function` che permettono di gestire le due modalità di identificazione di una funzione. Grazie alle funzioni `get_target_module` e `get_target_function` è infatti possibile ottenere i due valori basandosi su quelli valorizzati dall'utente. Nello specifico il nome del modulo può essere valorizzato in due modi: utilizzando il nome del modulo fornito dall'utente oppure il nome del target. Per quanto riguarda le funzioni, si utilizza il contenuto di `target_function`, se presente, altrimenti si utilizza un nome predefinito. Infine la CLI imposta il parametro `-artifact-prefix`, utilizzato da *libfuzzer* per salvare gli *artifacts*: dei file contenenti gli input che hanno fatto fallire l'esecuzione di un target.

Terminata l'esecuzione del fuzzer, viene controllato lo stato dell'esecuzione. Se l'esecuzione è terminata con successo il programma termina, se invece ci sono stati errori vengono visualizzati a schermo i messaggi di errore con informazioni aggiuntive, utili per replicare le condizioni che lo hanno causato.

4.6 Il comando FMT

Il comando `fmt` esegue un target passando un input specifico. Viene utilizzato per verificare il funzionamento del codice ed effettuare debug. Oltre agli argomenti descritti in precedenza per indicare la funzione target e per la configurazione del compilatore, accetta un parametro chiamato `<input>`, tramite il quale viene indicato il file che contiene i dati da passare alla funzione.

4.7 I comandi COVERAGE, CMIN e TMIN

Gli ultimi comandi che fanno parte della CLI riguardano la copertura del codice, ovvero delle . Il comando `coverage`, permette di generare dei report sulla copertura del codice, tuttavia sono ancora molto grezzi: hanno un funzionamento basilare e non lavorano a stretto contatto con il linguaggio di programmazione (come avveniva invece per il fuzzer di Rust al quale questo software è ispirato). Nello specifico, il comando `coverage`, esegue il target indicato generando un report sulla copertura basandosi quella singola esecuzione. Attualmente il fuzzer non supporta l'unione di più report per la creazione di un risultato più completo.

I comandi `cmin` e `tmin`, riguardano rispettivamente la minimizzazione del corpus e dei test case. Nel primo caso l'obiettivo è ridurre il numero di input necessari per mantenere invariata la copertura del codice, nel secondo caso l'obiettivo è quello di ridurre la dimensione dei singoli test-case mantenendo invariata la copertura del codice. Entrambe queste ottimizzazioni sono utili per migliorare l'efficienza del fuzzer, tuttavia in questa versione non sono molto efficaci per le limitazioni illustrate prima durante il calcolo del coverage.

5 Valutazione e sviluppi futuri

L'obiettivo di *move-fuzz* era lo sviluppo di un fuzzer per il linguaggio Move, in particolare la sua versione chiamata **Sui**, sviluppata dall'azienda Mysten Labs, con la quale ho collaborato durante la fase di sviluppo. Gli obiettivi che erano stati fissati inizialmente sono stati raggiunti: il fuzzer riesce ad eseguire programmi scritti in Move, generando la quasi totalità dei data-type presenti nel linguaggio. Sono state inoltre sviluppate funzioni di contorno per la gestione dei target del fuzzer e per la generazione dei report sulla copertura del codice. Il loro funzionamento, insieme ad altri aspetti del software, potrebbe essere di molto migliorato, la sezione 5.1 mostra quali potrebbero essere gli sviluppi futuri per rendere *move-fuzz* più completo.

Il progetto è stato valutato eseguendo dei test su alcuni progetti scritti in Move scaricati da GitHub utilizzando uno strumento chiamato GitHub Crawler[11], sviluppato da Luca Giacometti, un altro studente presso l'università di Trento. L'idea iniziale era quella di sfruttare il suo codice, che scarica tutte le repository GitHub scritte in Move, per sviluppare successivamente uno script che eseguisse il fuzzer su tutti quei progetti. Questo approccio è stato tuttavia scartato a causa di alcune difficoltà riscontrate durante lo sviluppo. Il problema principale riguarda le differenze tra le varie versioni di Move, soltanto i progetti scritti in Sui o Move nativo potevano essere presi in considerazione. Una volta selezionati quei progetti, mi sono però reso conto che quelli realmente rilevanti erano in un numero molto limitato e che spesso, le funzioni più interessanti non erano nemmeno eseguibili in automatico dal fuzzer, richiedevano tuttavia un approccio manuale. La maggior parte delle funzioni appartiene infatti a due categorie: quelle prive di parametri e quelle che accettano dei parametri per riferimento. Quelle appartenenti al primo gruppo, non accettando alcun parametro, non sono rilevanti per il fuzzer visto che non vi è alcun passaggio di dati. Quelle del secondo gruppo invece, richiedendo parametri per riferimento, non sono eseguibili dal fuzzer direttamente. È necessario creare un target specifico che richiede gli stessi dati senza utilizzare i riferimenti, per poi chiamare la funzione da testare in un secondo momento. L'approccio utilizzato per testare il progetto è stato, dopo accurate valutazioni, quello di eseguire i test in maniera manuale sui progetti più rilevanti e su alcune funzioni create appositamente per esaltare alcune funzionalità nello specifico.

Dai test è emerso che la tipologia di errori più comune è quella degli errori aritmetici dovuti da overflow, una tipologia di errori che il fuzzer riesce a riconoscere in tempi molto brevi. Dopo aver eseguito dei test su funzioni prese da progetti già esistenti, sono state sviluppati dei casi di test specifici per mettere alla prova il fuzzer su casi di test più complicati, seppur lontani dalla realtà. Un primo caso di test è stata la seguente moltiplicazione tra interi, nella quale i fattori generati dal fuzzer vengono divisi per un numero molto alto per diminuire le possibilità di generare overflow.

```
public fun fuzz_target(a: u256, b: u256) {
    (a/9999999999)*(b/999999999);
}
```

Casi di test simili a questo vengono tuttavia risolti in pochissimo tempo, per provare dunque a mettere in difficoltà il fuzzer è stato proposto uno scenario un po' più articolato.

```
module fuzz::fuzz_target_2 {
    use my_token::TokenTransfer::transfer_token;
    use my_token::TokenTransfer::Token;

    public fun fuzz_target(sender: Token, recipient: Token, amount: u64) {
        transfer_token(&mut sender, &mut recipient, amount);
```

```
    }
}

module my_token::TokenTransfer {
    // Define the token struct with an amount field.
    struct Token has copy, drop {
        amount: u64,
    }

    // Initializes a new token with a specified amount.
    public fun create_token(amount: u64): Token {
        Token { amount }
    }

    // Transfers tokens from the sender to the recipient.
    public fun transfer_token(sender: &mut Token, amount: u64) {
        // Subtract the amount from the sender
        sender.amount = sender.amount - amount;

        // Edge case that might cause an overflow under certain conditions
        if (amount > 10) {
            sender.amount = sender.amount + 1;
        };
    }
}
```

Anche questo esempio, che contiene una dimostrazione di come vengono importati moduli esterni all'interno di un target, l'errore viene trovato in tempi abbastanza brevi. Per mettere infine a dura prova il fuzzer è stato creato un target che va in errore solo quando una stringa passata in ingresso corrisponde alla stringa "fuzz". In questo caso tuttavia il fuzzer non riesce a trovare l'errore anche dopo un lungo tempo di esecuzione. In conclusione è possibile dire che il fuzzer è in grado di trovare errori e bug dovuti a calcoli aritmetici errati in tempi relativamente brevi. Se si tratta di *edge-cases* molto rari fa molta più fatica e spesso no riesce a trovare anomalie anche dopo lunghi tempi di esecuzione.

5.1 Sviluppi futuri

Il progetto *move-fuzz*, seppur funzionante in maniera discreta nella maggior parte dei casi, può essere migliorato sotto molti aspetti. Uno dei punti principali da migliorare è l'integrazione con il linguaggio Move stesso, in questa versione, infatti, il fuzzer non conosce molti dettagli riguardo al codice che sta eseguendo, genera i dati da passare ai vari target senza conoscere il loro reale contenuto. Ciò è dovuto soprattutto al fatto che Move viene eseguito all'interno di una macchina virtuale utilizzando un bytecode proprietario, viene dunque a mancare la stretta interazione che ci sarebbe con un linguaggio compilato come Rust. Un aspetto che migliorerebbe in modo significativo questo aspetto sarebbe un'analisi completa sulla copertura del codice. Utilizzando report sulla copertura più accurati e implementando delle logiche per sfruttarli al meglio si otterrebbe una notevole ottimizzazione degli input generati, oltre ad un utilizzo più efficace delle funzionalità di minimizzazione dei test-case.

Un altro aspetto nel quale il fuzzer potrebbe essere migliorato è il supporto alla generazione di strutture dati generiche, al momento non supportate. Il motivo per cui al momento non è possibile generarle è che servirebbe conoscere con precisione il *data-type* del loro contenuto. Al momento è necessario creare un target specifico che richiede la struttura dati in questione senza argomenti generici, da passare in un secondo momento alla funzione che si vuole testare. Ci potrebbero essere numerose soluzioni per aggirare questa problematica ma fin ora nessuna delle soluzioni prese in considerazione era convincente. La soluzione migliore sarebbe l'implementazione un meccanismo simile a quello già presente in Rust, nel quale è sufficiente implementare un trait per specificare come deve avvenire la

generazione di una struttura dati. Move, essendo progettato per la creazione di *smart contracts* e non essendo dunque un linguaggio di programmazione *general purpose*, non possiede meccanismi di ereditarietà tra strutture dati e nemmeno delle interfacce verso altri linguaggi di programmazione. Una soluzione nella quale ci sia la possibilità di implementare un metodo comune, implementato per tutti i tipi generabili dal fuzzer non è possibile senza determinate caratteristiche del linguaggio di programmazione.

6 Conclusioni

Affrontare un progetto come un fuzzer, soprattutto per un linguaggio di programmazione che non conoscevo prima d'ora, è stato una sfida su numerosi aspetti. Non avevo mai lavorato così a stretto contatto con la toolchain di un linguaggio di programmazione, inoltre lavorare su un progetto open source presenta numerose sfide, che non avevo mai effettuato prima d'ora. Lavorare all'interno di codice scritto da altri programmatori, comprenderne il significato e il funzionamento mi ha aiutato a migliorare le mie capacità di lettura del codice e di reverse-engeneering. Durante lo sviluppo di questo progetto ho inoltre consolidato le mie conoscenze sulla programmazione in Rust apprese negli scorsi anni, questo progetto mi ha permesso di approfondire il funzionamento delle FFI (Foreign Function Interface), ovvero tecniche per interfacciare codice scritto in linguaggi diversi (nel mio caso Rust e C++). Ho inoltre dovuto sviluppare un interfaccia a linea di comando, un aspetto che non avevo mai approfondito prima d'ora.

La realizzazione di questo progetto è stata in conclusione molto formativa sia per me sia come persona che come programmatore.

Bibliografia

[1] Cargo fuzz. https://github.com/rust-fuzz/cargo-fuzz. Progetto GitHub del fuzzer di Rust.

[2] Fuzzing. https://it.wikipedia.org/wiki/Fuzzing. Pagina Wikipedia riguardante il fuzzing.

[3] Libfuzzer. https://llvm.org/docs/LibFuzzer.html. Documentazione ufficiale della libreria libfuzzer.

[4] Llvm. https://llvm.org/. Documentazione ufficiale libreria LLVM.

[5] The move book. https://move-language.github.io/move/. Documentazione ufficiale del linguaggio Move.

[6] Move sui. https://github.com/move-language/move-sui. Progetto GitHub della toolchain di Move Sui.

[7] Rust fuzz book. https://rust-fuzz.github.io/book/introduction.html. Documentazione ufficiale Cargo Fuzz.

[8] Sui documentation. https://docs.sui.io/concepts/sui-move-concepts. Documentazione ufficiale Move Sui.

[9] Sui fuzzer. https://github.com/FuzzingLabs/sui-fuzzer/. Progetto GitHub dal quale si sono ispirati alcune parti del programma.

[10] Unsafe rust. https://doc.rust-lang.org/book/ch19-01-unsafe-rust.html?highlight=FFI#using-extern-functions-to-call-external-code. Pagina della documentazione ufficiale di rust dedicata al codice unsafe e alle FFI.

[11] Luca Giacometti. Github crawler. https://github.com/lucagiacometti19/confidentiality-analysis-benchmark/blob/main/src/github_api_utils/github_api_helper.rs. Link al progetto sviluppato da Luca Giacometti per scaricare da GitHub tutti i progetti scritti in Move.